Experiment 9

Student Name: Mohammad Ashaf Khan **UID: 23BCS11123**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:21/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design an online movie ticket booking system (similar to BookMyShow) that enables users to search movies, view show timings, select seats, and book tickets with high consistency and low latency.

2. **Objective**:

- To design a scalable architecture for a movie ticket booking platform.
- To enable real-time seat availability updates.
- To support seat selection, booking, and cancellation.
- To ensure high consistency to prevent double booking of seats.
- To implement user booking history and notification features.

3. **Tools Used:**

- **Frontend (ReactJS, HTML, CSS, JavaScript, WebSocket)** o Built responsive UI with real-time stock updates
- • **Backend (Node.js / Java / Spring Boot)** o Developed REST APIs for trading, authentication, and portfolio
- • **Database (PostgreSQL / MySQL)** o Managed financial transactions and user data
- • **Redis** o Used for caching stock prices and improving latency
- • **Message Systems / WebSocket** o Enabled real-time updates for stock prices and order status
- • **Deployment (AWS / GCP, Load Balancer, Docker)** o Scalable cloud infrastructure with containerization

4. **System Requirements:**

A. Functional Requirements

- Users should be able to register, login, and manage their profiles.
- Users should be able to search movies based on city, date, and genre.
- Users should be able to view theatres and show timings.
- Users should be able to select seats and initiate booking.

- The system should temporarily lock seats during booking.
- Users should be able to make payments using multiple payment methods.
- The system should confirm booking after successful payment.
- Users should receive booking confirmation notifications.
- Users should be able to view booking history and cancel tickets (if applicable).

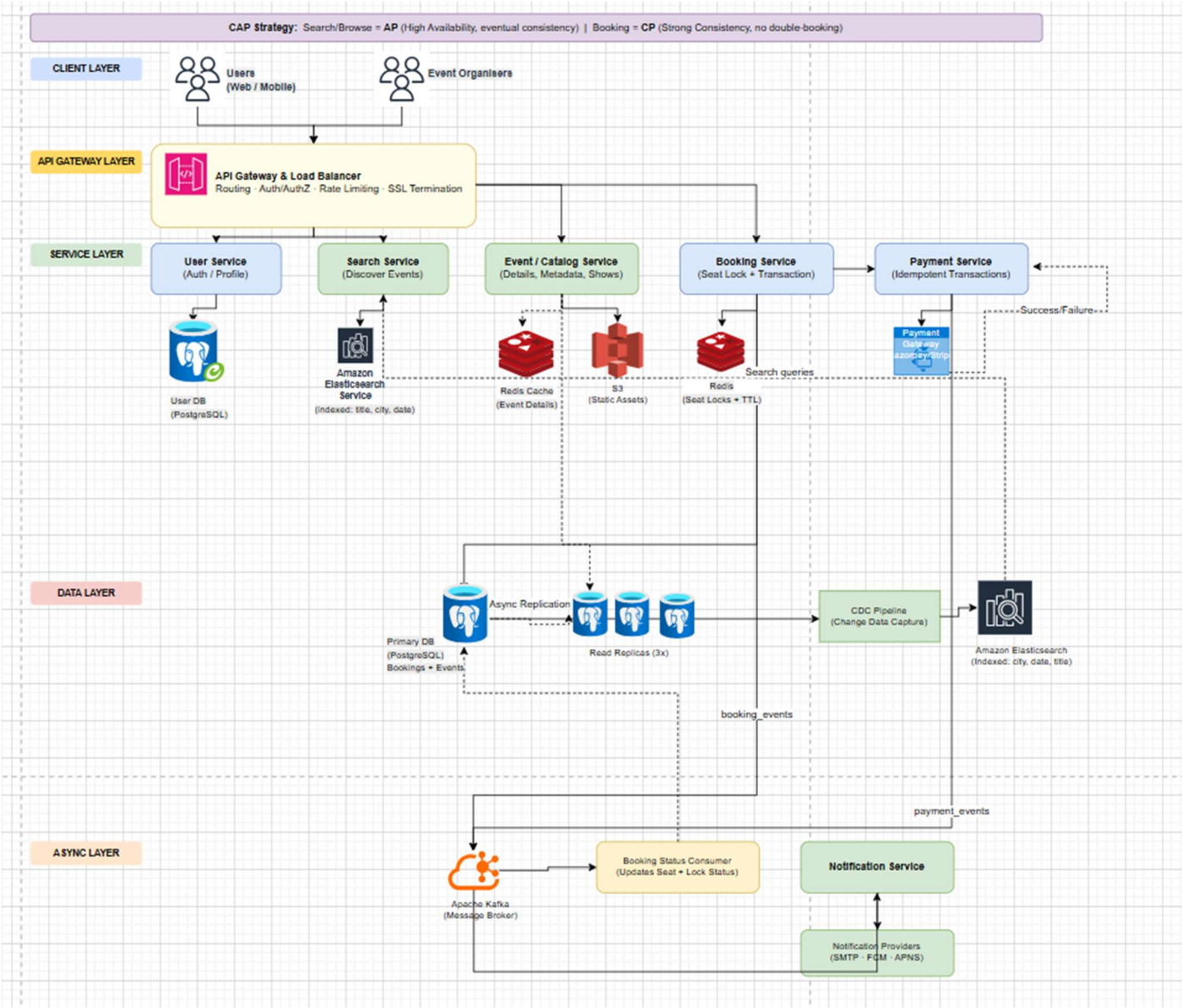
B. Non-Functional Requirements

- **Scalability**
  System should support millions of users during peak hours (e.g., new movie releases).

- **CAP Theorem**
  Consistency >>> Availability
  Seat booking must be strongly consistent to prevent double booking.
- **Latency:**
  Seat selection and booking response should be < 2–3 seconds
  Movie search latency should be < 100 ms

- **Reliability:**
  System must ensure no data loss and fault tolerance during booking and payment.

5. Low Level Design (LLD):



6. Scalability Solution

- Use Redis for seat locking and caching movie data
- Use load balancers to distribute incoming traffic
- Use microservices architecture (Auth, Movie, Booking, Payment, Notification)
- Use database replication and sharding for high scalability
- Use asynchronous queues (Kafka) for notifications
- Optimize read-heavy operations using caching
- Use horizontal scaling to handle high user traffic

7.Learning Outcomes (What I Have Learnt)

- Understood architecture of real-time booking systems
- Learned importance of consistency in seat booking
- Gained knowledge of caching and Redis-based locking
- Understood booking lifecycle (select → lock → pay → confirm)
- Learned how to design scalable and low-latency systems
- Understood trade-offs between availability and consistency